

Лекция по курсу «ОС и СП» №2

План лекции.

1. Управление процессами. Состояние процесса. Мультипрограммирование.
2. Алгоритмы планирования и диспетчеризации процессов.

Цели лекции.

Понять, что такое процесс, для чего он используется в ОС. Разобраться в управлении процессами. Разобраться, какие бывают состояния процессов, что такое мультипрограммирование. Разобрать алгоритмы планирования и диспетчеризации процессов. Ознакомиться с физической организацией жесткого диска, утилитами разбиения диска на разделы и форматирования их в разные системы.

Теоретическая часть лекции.

1. Управление процессами. Состояние процесса. Мультипрограммирование.

Процесс – это система действий, реализующая определенную функцию в вычислительной системе и оформленная так, что управляющая программа вычислительной системы может перераспределять ресурсы этой системы в целях обеспечения мультипрограммирования. Процесс (process) – это пользовательская программа при ее исполнении в компьютерной системе. Для выполнения процесса требуется ряд **ресурсов**, включая время процессора, память, файлы, устройства ввода-вывода, сетевые устройства и др.

Ресурс – средство вычислительной системы, которое может выделить процесс на определенный интервал времени. В общем смысле понимается некоторый запас материальных предметов либо энергетических, структурных или каких-либо других характеристик предмета в составе некоторого объекта.

Управление процессами. Процесс – это программа пользователя в ходе ее выполнения в компьютерной системе. ОС управляет работой процессов, их распределением по процессорам и ядрам системы, порядком их выполнения и размещения в памяти, их синхронизацией при параллельном решении частей одной и той же задачи разными процессами.

ОС отвечает за следующие действия, связанные с управлением процессами:

Создание и удаление процессов. При создании процесса необходимо создать в памяти соответствующие системные структуры (таблицу страниц, стек и др.). При удалении процесса память, занимаемая ими, освобождается, а также выполняется закрытие всех файлов и освобождение всех других ресурсов, которые использовал процесс, если последний не сделал этого явно.

Приостановка и возобновление процессов. Выполнение процесса приостанавливается при выполнении *синхронного ввода-вывода*, а также системного вызова или команды (типа **suspend**). Сразу отметим, что использовать подобные операции явной приостановки процессов следует с осторожностью, так как приостанавливаемый процесс может находиться в своей **критической секции** – выполнять обработку общего ресурса, к которому каждому процессу предоставляется монопольный доступ, так что при его приостановке возникает ситуация **тупика (deadlock)** – приостановленный процесс не может освободить ресурс, а конкурирующий процесс не может его получить. При приостановке процесса ОС сохраняет состояние его выполнения, а при возобновлении – восстанавливает.

Синхронизация процессов. Процессы работают параллельно и при этом конкурируют за общие ресурсы, а также должны в некоторые моменты вычислений ожидать наступления некоторых событий. Для предотвращения возможных конфликтов и несогласованностей, например, **race condition** - несогласованного доступа к общим данным, при котором один процесс читает старые данные, а другой их в этот же момент обновляет, - ОС предоставляет средства **синхронизации** (например, **семафоры** и **мониторы**).

Взаимодействие процессов. При своей параллельной работе процессам необходимо взаимодействие, с целью согласованного решения различных частей одной и той же задачи. Процессы могут взаимодействовать с помощью передачи **сообщений** друг другу, а также с помощью так называемых **условных переменных** и **рандеву** (все эти виды взаимодействия рассмотрены позже). ОС предоставляет все эти средства, в виде системных вызовов, для организации адекватного и удобного взаимодействия процессов.

Виды процессов:

- **системные:** являются частью ядра ОС и всегда находятся в оперативной памяти. Они не имеют соответствующих программ в виде исполняемых файлов и запускаются особым образом при инициализации ядра системы. Инструкции и данные этих процессов находятся в ядре системы, и они могут обращаться к функциям и данным, недоступным для остальных процессов.
- **демоны:** не интерактивные процессы, которые запускаются обычным образом (загрузка соответствующей программы) и выполняются в фоновом режиме. Обычно демоны запускаются при инициализации системы и обеспечивают работу различных подсистем (терминального доступа, печати, сетевого доступа и т.д.). Демоны не связаны ни с одним пользовательским сеансом и не могут непосредственно управляться пользователем. Большую часть времени демоны находятся в состоянии ожидания, пока какой-нибудь процесс не запросит определённую услугу.
- **прикладные:** К ним относятся все остальные процессы, выполняющиеся в системе. В большинстве случаев - это процессы, порожденные в рамках пользовательского сеанса работы. Пользовательские процессы могут выполняться как в интерактивном, так и в фоновом режиме, но время их жизни ограничено временем работы пользователя. При выходе из системы все пользовательские процессы уничтожаются. Интерактивные процессы монополюно владеют терминалом. И пока такой процесс не завершит работу, пользователь не может работать с другими процессами, если в функции интерактивного процесса не входит запуск на выполнение других программ.

Создание процесса

Причинами создания является:

- 1) Новое пакетное задание. Готовясь принять на обработку новое задание. ОС считывает очередные команды управления заданием.
- 2) Вход в систему в интерактивном режиме, когда в систему с терминала подключается новый пользователь.
- 3) Создание ОС процесса нужной для работы некоторой служебной ОС может создать процесс, для выполнения некоторой операции.
- 4) Порождение одного процесса другим с целью структуризации программы или использования возможности параллельных вычислений, программа может создавать другие процессы.

Когда создается новый процесс ОС создает структуры данных, используемых при управлении этим процессом и размещаемых в адресе пространства в основной памяти для процесса.

Завершение процесса

В любой системе должны быть средства, позволяющие определить, закончилось ли выполнение процесса. Для этого пакетное задание может включать в себя специальную команду или явный вызов служб ОС, приводящий к завершению процесса. При этом может генерироваться запрос на прерывание или выполнятся действие пользователя сообщаящего о том, что он покидает сеанс связи. Во всех случаях вызов специальных служб ОС завершает процесс.

Причин завершения достаточно много:

- 1) обычное завершение (процесс вызывает exit);
- 2) превышение лимита времени, отведенного программе;
- 3) недостаточный объем памяти;
- 4) выход за пределы отведенной области памяти;
- 5) ошибка защиты;
- 6) арифметическая ошибка (деление на ноль, переполнение);
- 7) изменение ожидания;
- 8) ошибка ввода/вывода;
- 9) неверная команда (счетчик команд переключается на счетчик данных);
- 10) команда с недопустимыми привилегиями;
- 11) непрерывное использование данных (часть данных или не инициализирована, или другого типа);
- 12) вмешательство оператора или ОС;
- 13) завершение родительского процесса;
- 14) запрос со стороны родительского процесса.

В классической схеме при создании процесса для него создается новое пространство виртуальной памяти, т.е. таблица страниц для отображения виртуальных адресов в физические, своя для каждого нового процесса. При этом расходуются значительные ресурсы. Если учесть, что в некоторых ОС каждая команда пользователя запускается как **отдельный процесс**, то становится понятным, насколько "дорога" операция создания процесса в классическом смысле. Поэтому еще в 1980-х гг. появилась концепция **облегченного процесса** (lightweight process) – выполняемого в том же пространстве виртуальной памяти, что и процесс-родитель. При создании нового облегченного процесса ОС создает для него только стек – системный резидентный массив в памяти, предназначенный для поддержки выполнения процедур процесса и хранящий их локальные данные и связующую информацию между ними.

В различных системах используются различные трактовки определения термина *процесс*. Рассмотрим уточнение понятия процесса.

Полновесные процессы – это процессы, выполняющиеся внутри защищенных участков памяти операционной системы, то есть имеющие собственные виртуальные адресные пространства для статических и динамических данных. В мультипрограммной среде управление такими процессами тесно связано с управлением и защитой памяти, поэтому переключение процессора с выполнения одного процесса на выполнение другого является достаточно дорогой операцией. В дальнейшем, используя термин процесс будем подразумевать полновесный процесс.

Легковесные процессы (их еще называют нитями или сопрограммами) не имеют собственных защищенных областей памяти. Они работают в мультипрограммном режиме одновременно с активировавшей их задачей и используют ее виртуальное адресное пространство, в котором им при создании выделяется участок памяти под динамические данные (стек), то есть они могут обладать собственными локальными данными. **Нить** описывается как обычная функция,

которая может использовать статические данные программы. Для одних операционных систем можно сказать, что нити являются некоторым аналогом процесса, а в других нити представляют собой части процессов. Таким образом, обобщая можно сказать – в любой операционной системе понятие «процесс» включает в себя следующее:

- исполняемый код;

- собственное адресное пространство, которое представляет собой совокупность виртуальных адресов, которые может использовать процесс;

- ресурсы системы, которые назначены процессу ОС;

- хотя бы одну выполняемую нить.

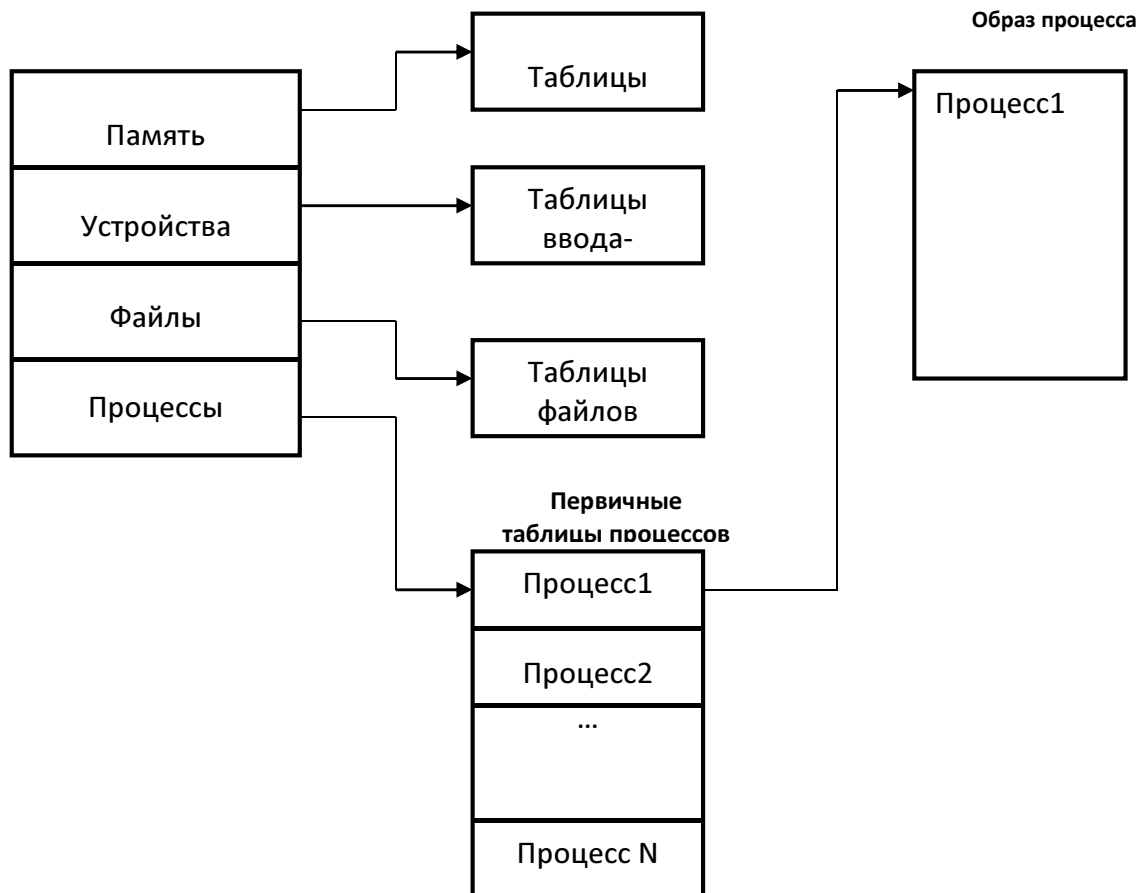
При этом подчеркнем – понятие процесса может включать в себя понятие исполняемой нити, т. е. **однонитевую организацию** – «один процесс – одна нить». В данном случае понятие процесса жестко связано с понятием отдельной и недоступной для других процессов виртуальной памяти. С другой стороны, в процессе может несколько нитей, т. е. процесс может представлять собой **многонитевую организацию**.

Нить также имеет *контекст*. **Контекст нити** – это информация, которая необходима ОС для того, чтобы продолжить выполнение прерванной нити. Контекст нити содержит текущее состояние регистров, стеков и индивидуальной области памяти, которая используется подсистемами и библиотеками. Как видно, в данном случае характеристики нити во многом аналогичны характеристикам процесса.

С точки зрения процесса, **нить** можно определить как *независимый поток управления*, выполняемый в контексте процесса.

Описание процессов

ОС можно представить как некий механизм, управляющий тем, как процессы используют системные ресурсы. И т.к. одна из задач ОС - управление процессами и ресурсами, то ОС должна располагать информацией о текущем состоянии каждого ресурса и процесса. Для этих целей ОС создает и поддерживает таблицы с информацией по каждому объекту управления. Общая структура таблиц:



Таблицы памяти используются для того, чтобы следить за основной и вторичной (виртуальной) памятью. Часть основной памяти резервируется для ОС, остальная доступна процессам.

Эти таблицы включают следующую информацию:

- 1) объем основной памяти, отведенной процессу;
- 2) объем вторичной или виртуальной памяти, отведенной процессу;
- 3) все атрибуты защиты блоков основной и виртуальной памяти;
- 4) всю информацию, необходимую для управления виртуальной памятью.

Таблицы ввода-вывода.

Используются для управления устройствами ввода-вывода и каналами компьютерной системы. В каждый момент времени устройство ввода-вывода может быть либо свободным, либо отданным в распоряжение какому-либо процессу. Если выполняется операция ввода-вывода, то должна быть информация о состоянии этой операции. Например, какие адреса ОП задействованы в этой операции, кто является отправителем и получателем отправляемой информации.

Таблицы файлов.

В них находится информация о существующих файлах, их расположение на магнитных носителях, текущем состоянии и других атрибутах. В ОС может быть специальная подсистема управления памятью.

Таблицы процессов.

Содержат сведения о процессах, располагая которыми ОС может управлять процессами.

Для эффективного управления информацией, которая хранится в таблицах, должны иметься перекрестные ссылки и находится в определенном взаимодействии, особенно в таблицах процессов по отношению к таблицам ресурсов.

Для создания этих таблиц ОС должна иметь информацию о конфигурации вычислительной системы: объем основной памяти, количество и вид устройств ввода/вывода, отнесение портов ввода/вывода к устройствам, информация о внешних устройствах хранения данных и т.п. Эти

данные могут создаваться с участием оператора или с помощью специальных программ определения конфигурации вычислительной системы.

Структура управления процессами

Для управления ОС должна знать, где находится процесс и атрибуты, нужные для управления процессами. Набор атрибутов, которые используются ОС, называется **управляющим блоком процесса**. Имеется понятие – **образ процесса**. В образ входит следующая информация.

1. **Данные пользователя** – это допускающая изменения часть пользовательского пространства, а также данные программы, пользовательский стек и модифицированный код
2. **Пользовательская программа** – программный код, который надо выполнить
3. **Системный стек**. С каждым процессом связаны один или несколько системных стеков. Стек используется для хранения параметров, адресов вызовов процедур и системных функций
4. **Управляющий блок процесса** – данные нужные ОС для управления процессом

Местонахождение образа процесса зависит от системы управления памятью. Но для управления процессом надо, чтобы часть его находилась в основной памяти. А чтобы запустить процесс, его надо полностью загрузить в основную и виртуальную память. Образы состоят из блоков, которые необязательно должны располагаться последовательно.

В большинстве современных систем поле разбивается на блоки или произвольного размера сегменты или одинакового размера страницы.

Атрибуты процессов

Типичные элементы управления блока процесса:

1. Идентификаторы процесса:

- идентификатор данного процесса;
- идентификатор родительского процесса;
- идентификатор пользователя.

2. Информация о состоянии процесса:

- регистры, доступные пользователю (те, к которым можно обращаться с помощью машинных команд);
- управляемые регистры и регистры состояния (те, которые управляют работой процесса). Должны быть счетчик команд, который содержит адрес очередной команды, коды условия, которые отражают результат выполнения арифметической или логической команды и информация о состоянии;
- указатели на стек – указывает на вершину стека, с процессом связывается один или несколько стеков.

3. Управляющая информация процесса:

а) информация по планированию и состоянию: состояние процесса, приоритет, может быть несколько значений приоритета: по умолчанию, текущий, максимально возможный. Информация, связанная с планированием: квант времени в течение, которого процесс выполняется при последовательном запуске. Информация о событии: идентификация события, инициализирующего продолжение работы события;

б) структуры данных: процесс может быть связан с другими процессами с помощью очереди, кольца или другими структурами. Для этого в управляемом блоке должны быть указатели на другие процессы;

в) обмен информацией между процессами: флаги, сигналы, сообщения;

г) привилегии процессов – это права доступа к определенным областям памяти, права выполнять определенные виды команд, возможности использования системных утилит;

д) управление памятью: это указатели на таблицы сегментов или страниц, описывающих распределение процессов в виртуальной памяти;

е) владение ресурсами и их использование: перечень открываемых файлов, сведения об исполнении процесса и других устройств ввода-вывода.

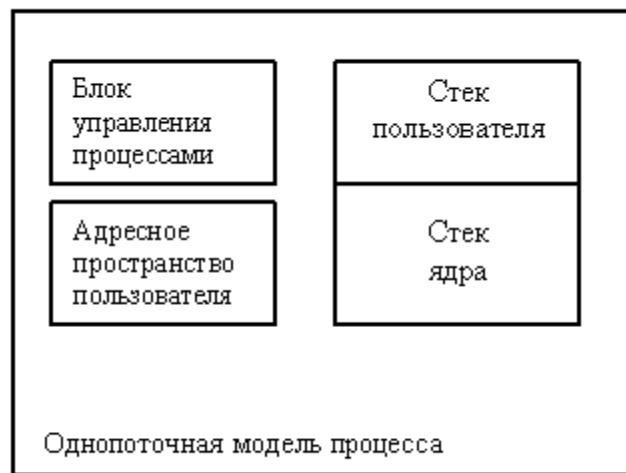
Процессы и потоки

Концепция процесса характеризуется двумя параметрами: владением ресурсами и планированием и выполнением.

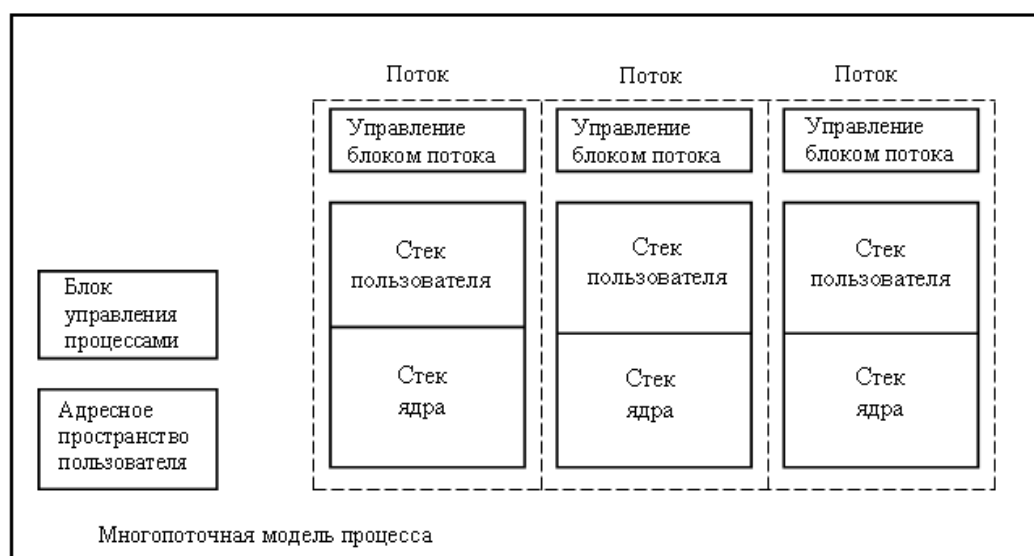
Многопоточность — это способность ОС поддерживать в рамках одного процесса выполнение нескольких потоков. Процесс, представляющий собой единый поток выполнения, называется однопоточным.

Есть различные конфигурации ОС:

- 1) один процесс — один поток (MS-DOS)
- 2) один процесс — несколько потоков (система типа Java)
- 3) несколько процессов — один поток (ОС UNIX на первых этапах)
- 4) несколько процессов и несколько потоков в процессе (Windows, OS/2, LINUX)



В многопоточной среде с каждым процессом также связаны управляющий блок и адресное пространство. Но для каждого потока создается свой отдельный стек и свой управляющий блок, в котором содержится значение регистров, приоритет и другая информация о состоянии потока.



Все потоки процесса разделяют между собой состояние и ресурсы этого процесса. Они находятся в адресном пространстве и имеют доступ к данным. Если один поток изменяет какие-то данные, то другие потоки во время своего доступа к данным могут отследить эти изменения. Если

один поток открывает файл, то другие потоки могут читать информацию из этого файла. Порождение новых потоков создаёт новую линию выполнения команд притом, что доступ идёт к одним и тем же данным.

С точки зрения производительности, использование потоков имеет следующие преимущества.

1. Создание нового потока в существующем процессе занимает на много меньше времени, чем создание нового процесса.

2. Поток можно завершить на много быстрее, чем процесс.

3. Переключение потоков в рамках одного процесса происходит на много быстрее, чем переключение процессов.

4. При использовании потоков повышается эффективность обмена информацией между двумя выполняющимися программами. В большинстве ОС обмен данными между независимыми процессами происходит с участием ядра. А так как потоки используют одно и тоже адресное пространство, то они могут обмениваться информацией без участия ядра.

Примером приложения, в котором можно применить поток, является файловый сервер. Так как серверу приходится обрабатывать большое количество запросов, то необходимо создание потоков. Если серверная программа работает на многопроцессорной машине, то на разных процессорах в рамках одного процесса могут выполняться одновременно несколько потоков.

Функциональность потоков

Потоки, подобно процессам, характеризуются состоянием выполнения и могут быть синхронизированы друг с другом. Основными состояниями потоков являются:

- состояние выполнения;
- состояние готовности;
- состояние блокировки.

С изменением состояния потоков связаны следующие действия:

1) Порождение.

Во время создания нового процесса, вместе с ним создаётся и поток этого процесса. В рамках одного и того же процесса поток может породить другой поток, определив его указатель команд и аргумент. Новый поток создаётся со своим собственным контекстом регистров и стековым пространством, после чего он помещается в очередь готовых к выполнению потоков.

2) Блокирование.

Если потоку нужно подождать, пока не наступит некоторое событие, то он блокируется. При этом сохраняется содержимое его пользовательских регистров, счётчик команд, указатель стека. После этого процессор может перейти к выполнению другого потока.

3) Разблокирование.

Когда наступает событие, которое ожидал заблокированный поток, тогда этот поток переходит в состояние готовности.

4) Выполнение.

Идет выполнение потока.

5) Завершение.

После завершения контексты регистров и стек потока удаляется.

6) Синхронизация потоков.

Так как все потоки используют одно и тоже адресное пространство и другие ресурсы, действия различных потоков надо синхронизировать, чтобы они не мешали друг другу и не повредили совместно используемые данные. При синхронизации потоков используются те же методы, что и при синхронизации процессов. Это значит, что решаются задачи потребитель-производитель, чтение-запись и другие.

Решением стало **мультипрограммирование – способ организации вычислительного процесса, при котором в памяти компьютера находилось одновременно несколько программ, попеременно выполняющихся на одном процессоре.** Эти усовершенствования значительно улучшили эффективность вычислительной системы. **Мультипрограммирование было реализовано в двух вариантах – в системах пакетной обработки и разделения времени.** Мультипрограммные системы пакетной обработки так же, как и их однопрограммные предшественники, имели своей целью обеспечение максимальной загрузки аппаратуры компьютера, однако решали эту задачу более эффективно. В мультипрограммном пакетном режиме процессор не простаивал, пока одна программа выполняла операцию ввода-вывода (как это происходило при последовательном выполнении программ в системах ранней пакетной обработки), а переключался на другую готовую к выполнению программу. В результате достигалась сбалансированная загрузка всех устройств компьютера, а, следовательно, увеличивалось число задач, решаемых в единицу времени.

В мультипрограммных системах пакетной обработки пользователь по-прежнему был лишен возможности интерактивного взаимодействия со своими программами. Для того, чтобы хотя бы частично вернуть пользователям ощущение непосредственного взаимодействия с компьютером, был разработан другой вариант мультипрограммных систем – системы разделения времени. Этот вариант рассчитан на многотерминальные системы, когда каждый пользователь работает за своим терминалом. В числе первых операционных систем разделения времени, разработанных в середине 60-х годов, были TSS/360 (компания IBM), CTSS и MULTICS (Массачусетский технологический институт совместно с Bell Labs и компанией General Electric).

Вариант мультипрограммирования, применяемый в системах разделения времени, был нацелен на создание для каждого отдельного пользователя иллюзии единоличного владения вычислительной машиной за счёт периодического выделения каждой программе своей доли процессорного времени. В системах разделения времени эффективность использования оборудования ниже, чем в системах пакетной обработки, что явилось платой за удобства работы пользователей. Многотерминальный режим использовался не только в системах разделения времени, но и в системах пакетной обработки. При этом не только оператор, но и все пользователи получали возможность формировать свои задания и управлять их выполнением со своего терминала. Такие ОС получили название систем удалённого ввода заданий. Терминальные комплексы могли располагаться на большом расстоянии от процессорных стоек, соединяясь с ними с помощью различных глобальных связей – модемных соединений телефонных сетей или выделенных каналов. Для поддержания удалённой работы терминалов в операционных системах появились специальные программные модули, реализующие различные (в то время, как правило, нестандартные) протоколы связи. Такие вычислительные системы с удалёнными терминалами, сохраняя централизованный характер обработки данных, в какой-то степени являлись прообразом современных сетей, а соответствующее системное программное обеспечение – прообразом сетевых операционных систем.

В компьютерах 60-х годов большую часть действий по организации вычислительного процесса взяла на себя операционная система. Реализация мультипрограммирования потребовала внесения очень важных изменений в аппаратуру компьютера, непосредственно направленных на поддержку нового способа организации вычислительного процесса. При разделении ресурсов компьютера между программами необходимо обеспечить быстрое переключение процессора с одной программы на другую, а также надёжно защитить коды и данные одной программы от непреднамеренной или преднамеренной порчи другой программы. В процессорах появился привилегированный и пользовательский режим работы, специальные регистры для быстрого переключения с одной программы на другую, средства защиты областей памяти, а также развитая система прерываний.

В привилегированном режиме, предназначенном для работы программных модулей операционной системы, процессор мог выполнять все команды, в том числе и те из них, которые позволяли осуществлять распределение и защиту ресурсов компьютера. Программам,

работающим в пользовательском режиме, некоторые команды процессора были недоступны. Таким образом, только ОС могла управлять аппаратными средствами и исполнять роль арбитра для пользовательских программ, которые выполнялись в непривилегированном, пользовательском режиме.

Система прерываний позволяла синхронизировать работу различных устройств компьютера, работающих параллельно и асинхронно, таких как каналы ввода-вывода, диски, принтеры и т. п.

Взаимодействие процессов

Если двум или более процессам необходимо взаимодействовать друг с другом, то они должны быть связаны, то есть иметь средства для обмена информацией. Предполагается, что процессы связаны слабо. Под этим подразумевается, что кроме достаточно редких моментов явной связи эти процессы рассматриваются как совершенно независимые друг от друга. В частности, не допускаются какие-либо предположения об относительных скоростях различных процессов.

2. Алгоритмизация и диспетчеризация процессов.

Планирование процессов включает в себя решение следующих задач:

1. определение момента времени для смены выполняемого процесса;
2. выбор процесса на выполнение из очереди готовых процессов;
3. переключение контекстов "старого" и "нового" процессов.

Первые две задачи решаются программными средствами, а последняя в значительной степени аппаратно.

Существует множество различных алгоритмов планирования процессов, по-разному решающих вышеперечисленные задачи, преследующих различные цели и обеспечивающих различное качество мультипрограммирования. Среди этого множества алгоритмов рассмотрим подробнее две группы наиболее часто встречающихся алгоритмов: алгоритмы, основанные на квантовании, и алгоритмы, основанные на приоритетах.

В соответствии с алгоритмами, основанными на квантовании, смена активного процесса происходит, если:

- процесс завершился и покинул систему,
- произошла ошибка,
- процесс перешел в состояние ОЖИДАНИЕ,
- исчерпан квант процессорного времени, отведенный данному процессу.

Процесс, который исчерпал свой квант, переводится в состояние ГОТОВНОСТЬ и ожидает, когда ему будет предоставлен новый квант процессорного времени, а на выполнение в соответствии с определенным правилом выбирается новый процесс из очереди готовых. Таким образом, ни один процесс не занимает процессор надолго, поэтому квантование широко используется в системах разделения времени. Граф состояний процесса, изображенный на рисунке 1, соответствует алгоритму планирования, основанному на квантовании.

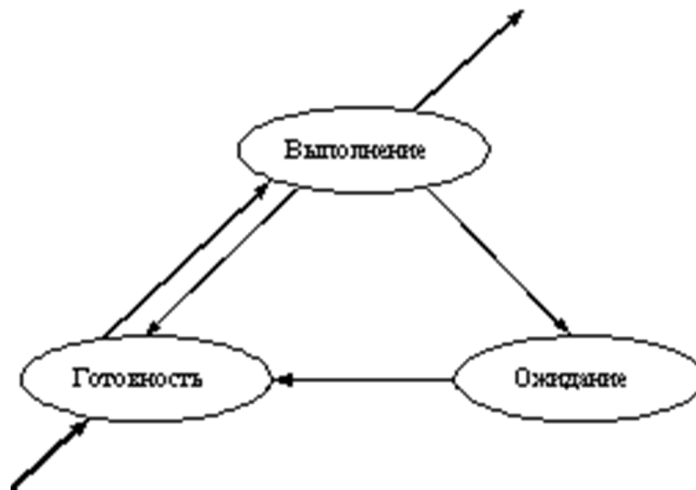


Рис. 1. Граф состояний процесса в многозадачной среде

Кванты, выделяемые процессам, могут быть одинаковыми для всех процессов или различными. Кванты, выделяемые одному процессу, могут быть фиксированной величины или изменяться в разные периоды жизни процесса. Процессы, которые не полностью использовали выделенный им квант (например, из-за ухода на выполнение операций ввода-вывода), могут получить или не получить компенсацию в виде привилегий при последующем обслуживании. По-разному может быть организована очередь готовых процессов: циклически, по правилу "первый пришел - первый обслужился" (FIFO) или по правилу "последний пришел - первый обслужился" (LIFO).

Другая группа алгоритмов использует понятие "приоритет" процесса. *Приоритет* - это число, характеризующее степень привилегированности процесса при использовании ресурсов вычислительной машины, в частности, процессорного времени: чем выше приоритет, тем выше привилегии.

Приоритет может выражаться целыми или дробными, положительным или отрицательным значением. Чем выше привилегии процесса, тем меньше времени он будет проводить в очередях. Приоритет может назначаться директивно администратором системы в зависимости от важности работы или внесенной платы, либо вычисляться самой ОС по определенным правилам, он может оставаться фиксированным на протяжении всей жизни процесса либо изменяться во времени в соответствии с некоторым законом. В последнем случае приоритеты называются динамическими.

Существует две разновидности приоритетных алгоритмов: алгоритмы, использующие относительные приоритеты, и алгоритмы, использующие абсолютные приоритеты.

В обоих случаях выбор процесса на выполнение из очереди готовых осуществляется одинаково: выбирается процесс, имеющий наивысший приоритет.

По-разному решается проблема определения момента смены активного процесса.

В системах с относительными приоритетами активный процесс выполняется до тех пор, пока он сам не покинет процессор, перейдя в состояние ОЖИДАНИЕ (или же произойдет ошибка, или процесс завершится).

В системах с абсолютными приоритетами выполнение активного процесса прерывается еще при одном условии: если в очереди готовых процессов появился процесс, приоритет которого выше приоритета активного процесса. В этом случае прерванный процесс переходит в состояние готовности. На рисунке 2 показаны графы состояний процесса для алгоритмов с относительными (а) и абсолютными (б) приоритетами.

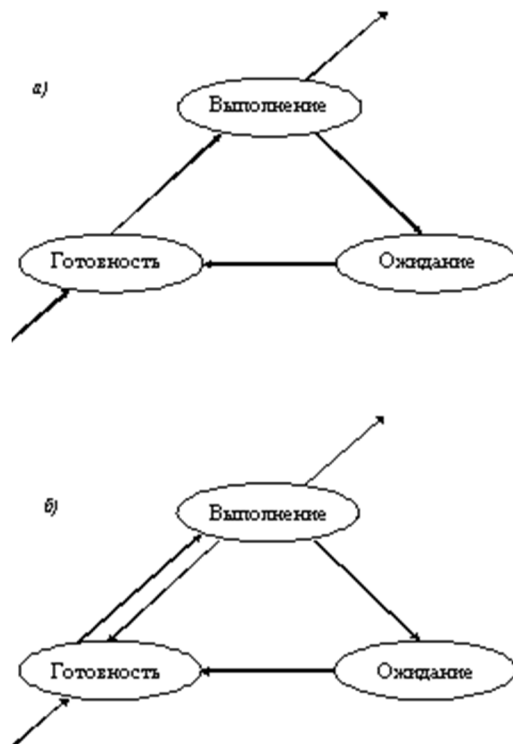


Рис. 2. Графы состояний процессов в системах
(а) с относительными приоритетами; (б) с абсолютными приоритетами

Во многих операционных системах алгоритмы планирования построены с использованием как квантования, так и приоритетов. Например, в основе планирования лежит квантование, но величина кванта и/или порядок выбора процесса из очереди готовых определяется приоритетами процессов.

Вытесняющие и невытесняющие алгоритмы планирования

Существует два основных типа процедур планирования процессов - вытесняющие (preemptive) и невытесняющие (non-preemptive).

Non-preemptive multitasking - невытесняющая многозадачность - это способ планирования процессов, при котором активный процесс выполняется до тех пор, пока он сам, по собственной инициативе, не отдаст управление планировщику операционной системы для того, чтобы тот выбрал из очереди другой, готовый к выполнению процесс.

Preemptive multitasking - вытесняющая многозадачность - это такой способ, при котором решение о переключении процессора с выполнения одного процесса на выполнение другого процесса принимается планировщиком операционной системы, а не самой активной задачей.

Понятия preemptive и non-preemptive иногда отождествляются с понятиями приоритетных и беспriorитетных дисциплин, что совершенно неверно, а также с понятиями абсолютных и относительных приоритетов, что неверно отчасти. Вытесняющая и невытесняющая многозадачность - это более широкие понятия, чем типы приоритетности. Приоритеты задач могут как использоваться, так и не использоваться и при вытесняющих, и при невытесняющих способах планирования. Так в случае использования приоритетов дисциплина относительных приоритетов может быть отнесена к классу систем с невытесняющей многозадачностью, а дисциплина абсолютных приоритетов - к классу систем с вытесняющей многозадачностью. А беспriorитетная дисциплина планирования, основанная на выделении равных квантов времени для всех задач, относится к вытесняющим алгоритмам.

Основным различием между preemptive и non-preemptive вариантами многозадачности является степень централизации механизма планирования задач. При вытесняющей многозадачности механизм планирования задач целиком сосредоточен в операционной системе, и программист пишет свое приложение, не заботясь о том, что оно будет выполняться параллельно

с другими задачами. При этом операционная система выполняет следующие функции: определяет момент снятия с выполнения активной задачи, запоминает ее контекст, выбирает из очереди готовых задач следующую и запускает ее на выполнение, загружая ее контекст.

При невытесняющей многозадачности механизм планирования распределен между системой и прикладными программами. Прикладная программа, получив управление от операционной системы, сама определяет момент завершения своей очередной итерации и передает управление ОС с помощью какого-либо системного вызова, а ОС формирует очереди задач и выбирает в соответствии с некоторым алгоритмом (например, с учетом приоритетов) следующую задачу на выполнение. Такой механизм создает проблемы как для пользователей, так и для разработчиков.

Для пользователей это означает, что управление системой теряется на произвольный период времени, который определяется приложением (а не пользователем). Если приложение тратит слишком много времени на выполнение какой-либо работы, например, на форматирование диска, пользователь не может переключиться с этой задачи на другую задачу, например, на текстовый редактор, в то время как форматирование продолжалось бы в фоновом режиме. Эта ситуация нежелательна, так как пользователи обычно не хотят долго ждать, когда машина завершит свою задачу.

Поэтому разработчики приложений для non-preemptive операционной среды, возлагая на себя функции планировщика, должны создавать приложения так, чтобы они выполняли свои задачи небольшими частями. Например, программа форматирования может отформатировать одну дорожку дискеты и вернуть управление системе. После выполнения других задач система возвратит управление программе форматирования, чтобы та отформатировала следующую дорожку. Подобный метод разделения времени между задачами работает, но он существенно затрудняет разработку программ и предъявляет повышенные требования к квалификации программиста. Программист должен обеспечить "дружественное" отношение своей программы к другим выполняемым одновременно с ней программам, достаточно часто отдавая им управление. Крайним проявлением "недружественности" приложения является его зависание, которое приводит к общему краху системы. В системах с вытесняющей многозадачностью такие ситуации, как правило, исключены, так как центральный планирующий механизм снимет зависшую задачу с выполнения.

Однако распределение функций планировщика между системой и приложениями не всегда является недостатком, а при определенных условиях может быть и преимуществом, потому что дает возможность разработчику приложений самому проектировать алгоритм планирования, наиболее подходящий для данного фиксированного набора задач. Так как разработчик сам определяет в программе момент времени отдачи управления, то при этом исключаются нерациональные прерывания программ в "неудобные" для них моменты времени. Кроме того, легко разрешаются проблемы совместного использования данных: задача во время каждой итерации использует их монопольно и уверена, что на протяжении этого периода никто другой не изменит эти данные. Существенным преимуществом non-preemptive систем является более высокая скорость переключения с задачи на задачу.

Примером эффективного использования невытесняющей многозадачности является файл-сервер NetWare, в котором, в значительной степени благодаря этому, достигнута высокая скорость выполнения файловых операций. Менее удачным оказалось использование невытесняющей многозадачности в операционной среде Windows 3.x.

Однако почти во всех современных операционных системах, ориентированных на высокопроизводительное выполнение приложений (UNIX, Windows NT, OS/2, VAX/VMS), реализована вытесняющая многозадачность. В последнее время дошла очередь и до ОС класса настольных систем, например, OS/2 Warp и Windows 95. Возможно в связи с этим вытесняющую многозадачность часто называют истинной многозадачностью.

Вопросы.